

Guía de Desarrollo de Componentes AVFenix

Manual Técnico del Lenguaje .avf, Compilador AFXC v9.0.0, Patrones Avanzados y SSR

RESUMEN EJECUTIVO Y ARQUITECTURA DUAL

AVFenix Types (.avf) combina en un solo archivo la definición tipada de esquemas persistentes para el CMS y componentes reactivos para el cliente. El compilador **AFXC v9.0.0** genera una salida triple: un manifiesto JSON (.schema.json) para el backend, un bundle ejecutable (.js) para General.JS (Reactive y Routing), y un Source Map V3 (.js.map).

1. Sintaxis Básica del Lenguaje (.avf)

Un archivo .avf se compone de cuatro bloques principales: **schema** (modelado de datos y UI administrativa), **component** (UI reactiva del cliente), **extension** (utilitarios globales) y **plugin** (interceptores del núcleo).

```
// Módulo: Artículo.avf
export schema Artículo {
  titulo: string @ui(widget: "text-input", label: "Título", required: true);
  contenido: text @ui(widget: "rich-editor");
  precio: number @validate(min: 0, max: 10000);
  autor: User @link(relation: "many-to-one");
}

export component ArtículoCard {
  state = { likes: 0 };
  onMount() { this.setState({ likes: this.props.initialLikes || 0 }); }

  template(state) {
    return (
      <div class="card">
        <h2>{this.props.titulo}</h2>
        <button onClick={() => this.setState({ likes: state.likes + 1 })}>
          Me gusta ({state.likes})
        </button>
      </div>
    );
  }
}
```

Decoradores Soportados en Esquemas

Decorador	Parámetros Soportados	Propósito en el CMS
@ui	widget, label, required, options, default	Configura la interfaz del panel de administración (inputs, selects, rich-text).
@validate	min, max, type (email/url), pattern	Aplica reglas de validación en tiempo de compilación y runtime.
@link	relation (one-to-one, many-to-one, etc.)	Establece relaciones ORM entre entidades del CMS.

2. Novedades y Capacidades del Compilador AFXC v9.0.0

El motor AFXC v9.0.0 incluye una arquitectura de producción diseñada para proyectos de gran escala:

- **Grafo de Dependencias Multi-Archivo:** Procesa sentencias `import { Specifier } from './path.avf'` y previene dependencias circulares recursivas.
- **Lexer AST con Línea y Columna:** Ubica exactamente los errores sintácticos emitiendo reportes formateados con punteros visuales (^).
- **AST Static Hoisting:** Detecta nodos JSX estáticos y los eleva a constantes globales fuera de `template()`, reduciendo el trabajo de `diff()` y `patch()` en `reactive.general.js`.
- **Componentes Personalizados y Slots:** Diferencia etiquetas HTML nativas de componentes PascalCase e inyecta hijos dinámicos mediante `props.children`.
- **Type-Checker Estático Semántico:** Verifica relaciones entre entidades, tipos desconocidos y consistencia de rangos de validación antes de emitir archivos.
- **Soporte para Cláusulas Export y Alias:** Admite la sintaxis `export schema` y alias de importación (`import { Specifier as Alias }`).

3. Patrones Avanzados de Diseño en AVFenix

A continuación se detallan tres patrones de arquitectura recomendados para desacoplar lógica y maximizar la reutilización de componentes en el cliente:

A) Patrón Componentes Compuestos (Compound Components) y Slots

Permite construir componentes complejos (como modales, pestañas o acordeones) mediante subcomponentes que comparten contexto e inyectan su contenido a través de `props.children`.

```
// Componente Padre: Modal
export component Modal {
  template(state) {
    if (!this.props.open) return null;
    return (
      <div class="modal-overlay">
        <div class="modal-content">
          {this.props.children}
        </div>
      </div>
    );
  }
}

// Uso con Slots dinámicos
export component ModalApp {
  state = { show: false };
  template(state) {
    return (
      <div>
        <button onClick={() => this.setState({ show: true })}>Abrir Modal</button>
        <Modal open={state.show}>
          <h2>Título del Modal</h2>
          <p>Contenido inyectado vía props.children</p>
          <button onClick={() => this.setState({ show: false })}>Cerrar</button>
        </Modal>
      </div>
    );
  }
}
```

B) Estado Global Compartido con Reactive Stores y Context API

Para evitar la propagación excesiva de propiedades (*prop drilling*), AVFenix interactúa con *reactive.general.js* mediante almacenes reactivos compartidos.

```
// Extensión de Store Global
extension GlobalStore {
  createAuthStore() {
    return reactiv.createReactiveState({
      user: null,
      isAuthenticated: false
    });
  }
}
```

4. Renderizado en el Lado del Servidor (SSR) e Hidratación

AFXC v9.0.0 sienta las bases para el Renderizado en el Servidor (SSR) combinando la generación previa de HTML estático en Node.js con la hidratación reactiva transparente en el navegador.

Flujo de Trabajo SSR & Rehidratación:

- 1. Pre-render en Servidor Node.js:** El motor evalúa el método *template()* del componente con el estado inicial y genera la cadena HTML estática mediante *reactv.renderToString(vnode)*.
- 2. Inyección de Estado Inicial:** El servidor adjunta el estado inicial formateado como JSON dentro de una etiqueta de script predefinida (*window.__INITIAL_STATE__*).
- 3. Hidratación en el Navegador:** Al cargar la página, el bundle compilado por *afxc* ejecuta *new Componente(props, '#app-root')*. El Virtual DOM de *reactive.general.js* detecta los nodos DOM existentes, adhiere los escuchadores de eventos (event listeners) y sincroniza el estado sin provocar un parpadeo visual (FOUC).

VENTAJAS DE SSR EN AVFENIX

- **Indexación SEO Impecable:** El CMS entrega páginas HTML completamente renderizadas con metadatos indexables por buscadores.
- **Carga Inicial Ultra-Rápida (FCP):** El usuario visualiza el contenido de inmediato antes de que se complete la descarga del JavaScript cliente.
- **Hidratación Progresiva:** Los eventos interactivos se conectan granularmente sobre los elementos DOM existentes.

5. Comandos CLI y Automatización CI/CD

El compilador se integra mediante el archivo **afxc.config.json** y scripts en **package.json**:

```
# Comandos CLI oficiales de AFXC:
npx afxc init                # Genera archivo afxc.config.json por defecto
npx afxc check [entrada.avf] # Ejecuta análisis estático y Type-Checker sin emitir archivos
npx afxc build [entrada.avf] [out] # Compila el proyecto completo generando salida triple (.json, .js, .js.map)

# Comandos de paquete NPM:
npm run check                # Verificación semántica de código
npm test                     # Ejecución de la suite de pruebas automatizadas (afxc.test.js)
npm run build                 # Construcción de artefactos de producción
```